FOOTBALL PLAYER MANAGEMENT SYSTEM

Course: PR0192 – **Object-Oriented Programming**
Group No: 3
Location & Time: Ho Chi Minh, 20 June 2026

Group No: 3	
Group Members	Lê Văn Quang - SE211971 Trương Trí Đăng - SE211934 Bùi Minh Tiến - SE212028 Ngô Nhật Huy - SE212039
Instructor	Thân Thị Ngọc Vân
Class	SE2017

Milestone 3 — Inheritance & Polymorphism

1. Summary of New Additions

In this milestone, I created a class hierarchy using inheritance and implemented polymorphism by overriding methods. This helps the system handle different types of objects using a common parent class.

I added an abstract top-level class called ClubEntity. Player and ClubRecord inherit from ClubEntity. Player is then extended by StarPlayer and RegularPlayer, while ClubRecord is

extended by Contract, Salary, MatchRecord, and TrainingSession. Each concrete child class overrides getEntityType() and displayInfo() to provide its own behavior.

2. Class Hierarchy Design

The top parent class is ClubEntity. It stores the shared ID and declares two common abstract methods: getEntityType() and displayInfo().

The two direct child classes are Player and ClubRecord. Player contains common player information such as full name, age, nationality, position, shirt number, base salary, and status. ClubRecord groups the football-club records that share an ID but have different data fields.

Class	Direct Parent	Role / Unique Behavior
ClubEntity	None	Abstract top-level class. Stores the common ID and declares `getEntityType()` and `displayInfo()` for child classes.
Player	ClubEntity	Abstract parent class for all player types. Stores common player information.
StarPlayer	Player	Calculates monthly salary with a 20% bonus and displays Star Player information.
RegularPlayer	Player	Calculates monthly salary with a 5% bonus and displays Regular Player information.
ClubRecord	ClubEntity	Abstract parent class that groups football club record objects.
Contract	ClubRecord	Stores contract dates and contract status.
Salary	ClubRecord	Stores player monthly salary, performance bonus, and total salary.
MatchRecord	ClubRecord	Stores match date, opponent team, and match type.
TrainingSession	ClubRecord	Stores training date, location, and topic.

Table 1. Class hierarchy and responsibilities

3. UML Hierarchy Diagram

Figure 1 shows the inheritance hierarchy implemented in the system. The arrows represent inheritance from a child class to its parent class.

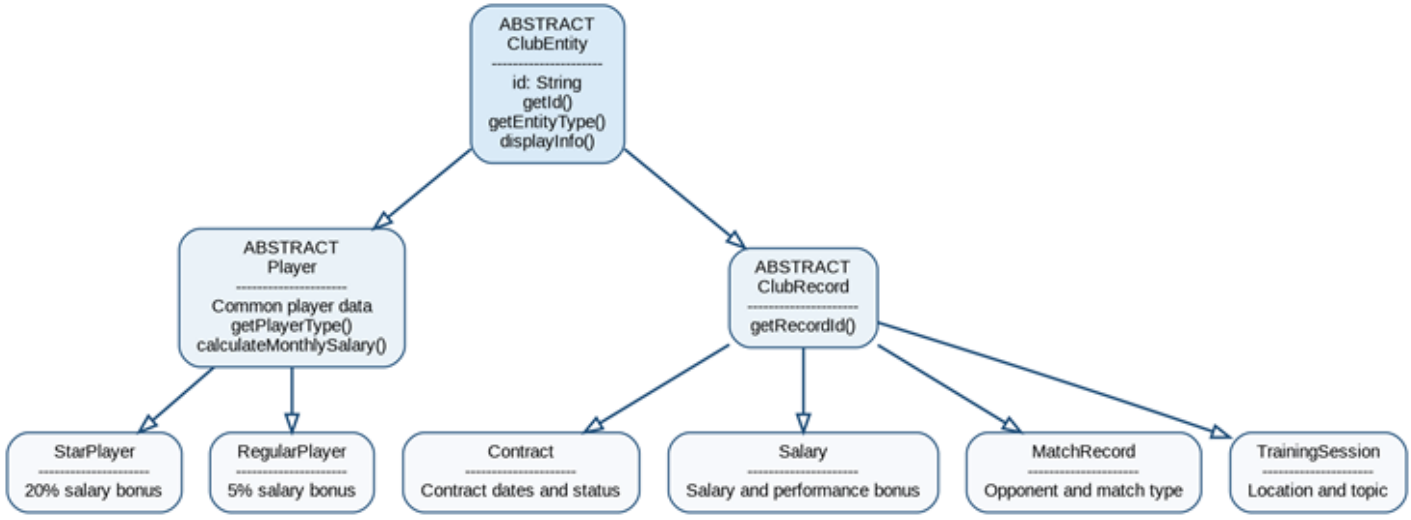

Figure 1. UML hierarchy diagram with inheritance relationships

4. Polymorphism Demonstration

Polymorphism is demonstrated in the existing PlayerList class. The list uses the parent type Player, so it can store both StarPlayer and RegularPlayer objects in the same ArrayList.

```
private ArrayList<Player> players;
```

When a user adds a player, the program asks for the player type and creates the correct child object. The return type is Player, even though the real object is StarPlayer or RegularPlayer.

```
private Player createPlayerByType(Scanner sc) {
    while (true) {
        System.out.print("Player Type (Star/Regular): ");
        String playerType = sc.nextLine().trim();

        if (playerType.equalsIgnoreCase("Star")) {
          return new StarPlayer();
        }
        if (playerType.equalsIgnoreCase("Regular")) {
          return new RegularPlayer();
        }
        System.out.println("Player type must be Star or Regular.");
    }
 }
```

The display function then uses the same method call for every object in the list.

```
for (int i = 0; i < players.size(); i++) {
    Player player = players.get(i);
    player.displayInfo();
}
```

Although the code calls player.displayInfo() in the same way, Java chooses the correct overridden method based on the real object at runtime. If the object is StarPlayer, Java runs StarPlayer.displayInfo(). If the object is RegularPlayer, Java runs RegularPlayer.displayInfo(). This runtime selection is called dynamic dispatch.

4.1 Overridden Methods in Player Subclasses

StarPlayer and RegularPlayer override the same abstract method from Player, but each subclass uses its own salary rule.

```
// StarPlayer.java
@Override
public double calculateMonthlySalary() {
    return getBaseSalary() * (1 + STAR_BONUS_RATE);
}

// RegularPlayer.java
@Override
public double calculateMonthlySalary() {
    return getBaseSalary() * (1 + REGULAR_BONUS_RATE);
}
```

For example, a StarPlayer with a base salary of 1000 receives 1200 after a 20% bonus. A RegularPlayer with a base salary of 800 receives 840 after a 5% bonus. The method name is the same, but the result depends on the actual object type.

4.2 Dynamic Dispatch in the Salary Module

The SalaryList class also uses polymorphism in a real feature of the system. It finds a player using the parent type Player and calls calculateMonthlySalary() without writing separate if-else statements for StarPlayer and RegularPlayer.

```
Player player = playerList.findPlayerById(salary.getPlayerId());

salary.setPlayerMonthlySalary(
        player.calculateMonthlySalary()
);
```

If the real object is StarPlayer, Java applies the 20% bonus rule. If the real object is RegularPlayer, Java applies the 5% bonus rule. This makes the salary module easier to extend because a future player type can provide its own calculation method.

4.3 Test Output Evidence

```
--- Player 3 ---
Player ID: 003
Full Name: Le Van Than
Age: 26
Nationality: VNM
Position: ST
Shirt Number: 12
Base Salary: 1654560.0
Status: active
Player Type: Regular
Regular Bonus Rate: 5%
Calculated Monthly Salary: 1737288.0
Benefit: Standard team training program.

--- Player 1 ---
Player ID: 001
Full Name: Nguyen Van Muoi
Age: 24
Nationality: VNM
Position: RW
Shirt Number: 11
Base Salary: 1.354E7
Status: active
Player Type: Star
Star Bonus Rate: 20%
Calculated Monthly Salary: 1.6248E7
Benefit: Priority training and promotion activities.
```

5. Algorithmic Thinking Evidence

I designed the overridden methods by analyzing how each subclass behaves differently. First, I identified the common features shared by the main objects: every object has an ID and can display its information. These features were placed in ClubEntity.

Next, I separated the objects into two groups. Player stores common player data, while ClubRecord groups records such as Contract, Salary, MatchRecord, and TrainingSession. This prevents duplicated ID code and makes the class hierarchy clearer.

For the player subclasses, I created two different salary algorithms. StarPlayer multiplies base salary by 1.20, while RegularPlayer multiplies base salary by 1.05. The salary module then calls

calculateMonthlySalary() through a Player reference, so the correct algorithm is selected automatically according to the object type.

6. Reflection

In this milestone, I learned how inheritance reduces duplicated code by moving common attributes and methods into parent classes. I learned that abstract classes are useful when a general class should not be created directly. I also understood how method overriding lets each child class provide its own behavior. Polymorphism allows one parent reference to work with many different child objects. Finally, I learned that this design makes the salary module easier to extend because a new player type can add its own salary rule without changing the existing salary logic.